Lập trình Shell Script cơ bản trên Debian

Mục tiêu

Sau bài thực hành, sinh viên có thể:

- Tạo, phân quyền và thực thi một chương trình shell script.
- Sử dụng biến, tham số dòng lệnh, phép toán số học và phép toán logic.
- Viết rẽ nhánh `if`, `case` và kiểm tra điều kiện trên tập tin, thư mục.
- Sử dụng các vòng lặp `for`, `while`, `until`.
- Viết các script đơn giản xử lý tập tin, thư mục phục vụ công việc quản trị hệ thống.

Chuẩn bị

- Một máy Debian đã đăng nhập bằng tài khoản thường (không dùng `root` trừ khi bài yêu cầu).
- Trình soạn thảo `nano` hoặc `vim`. Cài nano nếu chưa có: `sudo apt install nano`.
- Tạo thư mục làm việc cho toàn bộ bài lab:

```
mkdir -p ~/lab_shell
cd ~/lab_shell
pwd
```

> **Lưu ý:** Trên Debian, `/bin/sh` là `dash`, không phải `bash`. Vì vậy mọi script trong bài này đều khai báo `#!/bin/bash` để dùng được các tính năng như `[[ ]]`, mảng, `(( ))`.

Phần 1. Tạo và thực thi chương trình shell

1.1. Shell script là gì?

Shell script là tập tin văn bản chứa các lệnh shell, được thông dịch và chạy tuần tự. Dòng đầu tiên `#!/bin/bash` (gọi là *shebang*) cho hệ thống biết dùng trình thông dịch nào. Ký tự `#` ở đầu dòng (trừ shebang) là chú thích.

1.2. Bài thực hành 1.1: Chương trình đầu tiên

Bước 1. Tạo tập tin:

```
nano hello.sh
```

Bước 2. Nhập nội dung:

```
#!/bin/bash
# Chuong trinh dau tien
echo "Xin chao, $USER!"
echo "Hom nay la: $(date +%d/%m/%Y)"
echo "Ban dang o thu muc: $(pwd)"
```

Bước 3. Lưu (Ctrl+O, Enter) và thoát (Ctrl+X).

1.3. Các cách thực thi

Cách 1: Gọi trực tiếp bằng bash (không cần quyền thực thi)

```
bash hello.sh
```

Cách 2: Cấp quyền thực thi rồi chạy

```
ls -l hello.sh          # xem quyền hiện tại: -rw-r--r--
chmod +x hello.sh       # thêm quyền thực thi
ls -l hello.sh          # -rwxr-xr-x
./hello.sh
```

Cách 3: Chạy từ bất kỳ đâu bằng cách đưa vào PATH

```
mkdir -p ~/bin
cp hello.sh ~/bin/
export PATH=$PATH:~/bin
hello.sh
```

Câu hỏi 1: Vì sao gõ `hello.sh` mà không có `./` thì báo lỗi `command not found` khi chưa thêm `~/bin` vào PATH?

1.4. Tham số dòng lệnh và mã thoát

Biến	Ý nghĩa
`$0`	Tên script
`$1`, `$2`, …	Tham số thứ 1, 2, …
`$#`	Số lượng tham số
`$@`	Danh sách tất cả tham số
`$?`	Mã thoát (exit code) của lệnh vừa chạy; `0` là thành công
`$$`	PID của tiến trình script

Bài thực hành 1.2

```
nano thamso.sh
```

```
#!/bin/bash
echo "Ten script      : $0"
echo "Tham so thu 1   : $1"
echo "Tham so thu 2   : $2"
echo "So tham so      : $#"
```

```
echo "Tat ca tham so  : $@"
exit 0

chmod +x thamso.sh
./thamso.sh debian linux server
echo $?
```

Phần 2. Biến, phép toán số học, phép toán logic

2.1. Biến

- Gán giá trị: **không có khoảng trắng** quanh dấu =. Đọc giá trị bằng `$ten_bien`.
- Tên biến phân biệt hoa thường, chỉ gồm chữ, số, dấu gạch dưới và không bắt đầu bằng số.
- Dấu nháy kép `" "` cho phép thay thế biến; nháy đơn `' '` giữ nguyên văn bản.
- `$(lệnh)` lấy kết quả in ra của lệnh gán vào biến.
- `read` đọc dữ liệu nhập từ bàn phím.

Bài thực hành 2.1

```
nano bien.sh

#!/bin/bash
ten="Debian"
phienban=12
echo "He dieu hanh: $ten $phienban"
echo 'Nhay don: $ten khong duoc thay the'

ngay=$(date +%Y-%m-%d)
souser=$(wc -l < /etc/passwd)
echo "Ngay: $ngay, so dong trong /etc/passwd: $souser"

read -p "Nhap ten cua ban: " hoten
echo "Chao $hoten"

# Bien moi truong va export
export MOITRUONG="lab_shell"
bash -c 'echo "Tu shell con: $MOITRUONG"'
```

Quan sát: Thử bỏ `export` ở biến `MOITRUONG`, chạy lại và giải thích sự khác biệt.

2.2. Phép toán số học

Bash chỉ xử lý số nguyên. Có các cách tính sau:

Cú pháp	Ví dụ
`$(( biểu_thức ))`	`kq=$((a + b))`
`let`	`let kq=a*b`

Cú pháp	Ví dụ
`expr`	`kq=$(expr $a + $b)`
`bc` (số thực)	`echo "scale=2; 10/3" | bc`

Các toán tử: `+  -  *  /  %  **` (lũy thừa), cùng `++`, `--`, `+=`, `-=` trong `(( ))`.

Bài thực hành 2.2: Máy tính đơn giản

```
#!/bin/bash
# tinh.sh - nhan 2 so tu tham so
if [ $# -ne 2 ]; then
    echo "Cach dung: $0 so1 so2"
    exit 1
fi
a=$1
b=$2
echo "$a + $b = $((a + b))"
echo "$a - $b = $((a - b))"
echo "$a * $b = $((a * b))"
echo "$a / $b = $((a / b))   (chia nguyen)"
echo "$a % $b = $((a % b))"
echo "$a ** 2 = $((a ** 2))"
echo "Chia so thuc: $(echo "scale=3; $a / $b" | bc)"

chmod +x tinh.sh
./tinh.sh 17 5
./tinh.sh 17
```

Nếu chưa có `bc`, cài bằng `sudo apt install bc`.

2.3. Phép toán so sánh và logic

Lệnh `test` hay `[ ... ]` (và `[[ ... ]]` trong bash) trả về mã thoát `0` nếu đúng, khác `0` nếu sai.

So sánh số nguyên

Toán tử	Ý nghĩa
`-eq`	bằng
`-ne`	khác
`-gt`	lớn hơn
`-ge`	lớn hơn hoặc bằng
`-lt`	nhỏ hơn
`-le`	nhỏ hơn hoặc bằng

So sánh chuỗi

Toán tử	Ý nghĩa

Toán tử	Ý nghĩa
= hoặc ==	bằng nhau
!=	khác nhau
-z "$s"	chuỗi rỗng
-n "$s"	chuỗi không rỗng

Toán tử logic

Toán tử	Dùng trong [ ]	Dùng trong [[ ]]	Giữa các lệnh
VÀ (AND)	-a	&&	&&
HOẶC (OR)	-o	||	||
PHỦ ĐỊNH (NOT)	!	!	!

Bài thực hành 2.3

Chạy từng lệnh trực tiếp trên terminal và ghi lại kết quả:

```
[ 5 -gt 3 ]; echo $?
[ 5 -lt 3 ]; echo $?
[ "abc" = "abc" ]; echo $?
[ -z "" ]; echo $?
[[ 5 -gt 3 && 2 -gt 1 ]]; echo $?
[[ 5 -gt 3 && 2 -gt 9 ]]; echo $?
[[ 5 -lt 3 || 2 -gt 1 ]]; echo $?

# Toan tu giua cac lenh
mkdir -p test_dir && echo "Tao thu muc thanh cong"
ls /khong_ton_tai 2>/dev/null || echo "Thu muc khong ton tai"
```

Câu hỏi 2: Vì sao nên đặt biến trong dấu nháy kép khi so sánh chuỗi, ví dụ `[ "$s" = "abc" ]`?

Phần 3. Rẽ nhánh và kiểm tra điều kiện trên tập tin

3.1. Cấu trúc if

```
if [ điều_kiện ]; then
    lệnh
elif [ điều_kiện_khác ]; then
    lệnh
else
    lệnh
fi
```

Chú ý: có khoảng trắng sau `[` và trước `]`; `then` đặt cùng dòng thì cần dấu `;` phía trước.

Bài thực hành 3.1: Phân loại điểm

```
#!/bin/bash
read -p "Nhap diem (0-10): " diem

if [ "$diem" -lt 0 ] || [ "$diem" -gt 10 ]; then
    echo "Diem khong hop le"
    exit 1
elif [ "$diem" -ge 8 ]; then
    echo "Loai Gioi"
elif [ "$diem" -ge 6 ]; then
    echo "Loai Kha"
elif [ "$diem" -ge 5 ]; then
    echo "Loai Trung binh"
else
    echo "Loai Yeu"
fi
```

3.2. Kiểm tra điều kiện trên tập tin và thư mục

Toán tử	Đúng khi
`-e file`	tồn tại (tập tin hoặc thư mục)
`-f file`	tồn tại và là tập tin thường
`-d file`	tồn tại và là thư mục
`-r file`	có quyền đọc
`-w file`	có quyền ghi
`-x file`	có quyền thực thi
`-s file`	tồn tại và kích thước lớn hơn 0
`-L file`	là liên kết mềm (symbolic link)
`f1 -nt f2`	`f1` mới hơn `f2`
`f1 -ot f2`	`f1` cũ hơn `f2`

Bài thực hành 3.2: Kiểm tra đường dẫn

```
#!/bin/bash
# kiemtra.sh
if [ $# -ne 1 ]; then
    echo "Cach dung: $0 duong_dan"
    exit 1
fi

duongdan="$1"

if [ ! -e "$duongdan" ]; then
    echo "'$duongdan' khong ton tai"
    exit 2
fi
```

```
if [ -f "$duongdan" ]; then
    echo "'$duongdan' la tap tin thuong"
elif [ -d "$duongdan" ]; then
    echo "'$duongdan' la thu muc"
else
    echo "'$duongdan' la loai tap tin khac"
fi

[ -r "$duongdan" ] && echo " - Co quyen doc"
[ -w "$duongdan" ] && echo " - Co quyen ghi"
[ -x "$duongdan" ] && echo " - Co quyen thuc thi"
[ -s "$duongdan" ] && echo " - Khong rong"

chmod +x kiemtra.sh
./kiemtra.sh /etc/passwd
./kiemtra.sh /etc
./kiemtra.sh /etc/shadow
./kiemtra.sh /khong/co
```

Quan sát: Vì sao `/etc/shadow` không có quyền đọc với tài khoản thường?

3.3. Cấu trúc case

`case` phù hợp khi so khớp một giá trị với nhiều mẫu, thường dùng để làm menu hoặc xử lý tham số.

Bài thực hành 3.3: Menu quản trị đơn giản

```
#!/bin/bash
# menu.sh
echo "===== MENU ====="
echo "1. Xem ngay gio he thong"
echo "2. Xem dung luong dia"
echo "3. Xem dia chi IP"
echo "4. Xem nguoi dung dang dang nhap"
echo "q. Thoat"
read -p "Chon: " chon

case $chon in
    1) date ;;
    2) df -h ;;
    3) ip -br addr ;;
    4) who ;;
    q|Q) echo "Tam biet"; exit 0 ;;
    *) echo "Lua chon khong hop le" ;;
esac
```

Phần 4. Vòng lặp

4.1. Vòng lặp for

Dạng danh sách:

```
for bien in gia_tri_1 gia_tri_2 gia_tri_3; do
    lệnh
done
```

Dạng kiểu C:

```
for (( i=1; i<=10; i++ )); do
    lệnh
done
```

Bài thực hành 4.1

```
#!/bin/bash
# vonglap_for.sh
echo "--- Duyet danh sach ---"
for ten in An Binh Chi; do
    echo "Xin chao $ten"
done

echo "--- Duyet day so ---"
for i in {1..5}; do
    echo "So $i"
done

echo "--- for kieu C: tong 1..100 ---"
tong=0
for (( i=1; i<=100; i++ )); do
    tong=$((tong + i))
done
echo "Tong = $tong"

echo "--- Duyet tap tin trong /etc bat dau bang 'host' ---"
for f in /etc/host*; do
    echo "$f"
done
```

4.2. Vòng lặp while và until

- `while`: lặp **khi** điều kiện còn đúng.
- `until`: lặp **cho đến khi** điều kiện đúng.
- `break` thoát khỏi vòng lặp; `continue` bỏ qua phần còn lại của lần lặp hiện tại.

Bài thực hành 4.2

```
#!/bin/bash
# vonglap_while.sh
```

```
# while: dem nguoc
n=5
while [ $n -gt 0 ]; do
    echo "Dem nguoc: $n"
    n=$((n - 1))
done

# until: lap den khi nhap dung mat khau
until [ "$matkhau" = "debian" ]; do
    read -s -p "Nhap mat khau (debian): " matkhau
    echo
done
echo "Dang nhap thanh cong"

# continue va break: in so le, dung khi gap 9
for i in {1..20}; do
    [ $((i % 2)) -eq 0 ] && continue
    echo "So le: $i"
    [ $i -ge 9 ] && break
done
```

4.3. Đọc tập tin từng dòng

```
#!/bin/bash
# doc_passwd.sh: liet ke ten user va shell
while IFS=: read -r user pass uid gid mota home shell; do
    echo "User: $user | UID: $uid | Shell: $shell"
done < /etc/passwd
```

Câu hỏi 3: Dấu `IFS=:` trong ví dụ trên có tác dụng gì? Sửa script để chỉ in những tài khoản có `UID >= 1000`.

Phần 5. Ví dụ xử lý tập tin, thư mục

Chuẩn bị dữ liệu mẫu:

```
cd ~/lab_shell
mkdir -p data
cd data
touch bao_cao_{1..5}.txt anh_{1..3}.jpg ghi_chu.log
echo "noi dung thu" > bao_cao_1.txt
cd ..
ls data
```

Ví dụ 5.1: Đếm tập tin và thư mục con

```
#!/bin/bash
# demfile.sh
thumuc=${1:-.}          # neu khong co tham so thi dung thu muc hien tai
```

```
if [ ! -d "$thumuc" ]; then
    echo "Loi: '$thumuc' khong phai thu muc"
    exit 1
fi

sofile=0
sodir=0
for muc in "$thumuc"/*; do
    if [ -f "$muc" ]; then
        sofile=$((sofile + 1))
    elif [ -d "$muc" ]; then
        sodir=$((sodir + 1))
    fi
done
echo "Thu muc: $thumuc"
echo "So tap tin   : $sofile"
echo "So thu muc con: $sodir"

chmod +x demfile.sh
./demfile.sh data
./demfile.sh /etc
```

Ví dụ 5.2: Đổi phần mở rộng hàng loạt

Đổi tất cả tập tin `.txt` trong `data` thành `.bak`:

```
#!/bin/bash
# doiduoi.sh
thumuc="$1"
cu="$2"
moi="$3"

if [ $# -ne 3 ]; then
    echo "Cach dung: $0 thu_muc duoi_cu duoi_moi"
    echo "Vi du   : $0 data txt bak"
    exit 1
fi

for f in "$thumuc"/*."$cu"; do
    [ -e "$f" ] || { echo "Khong co tap tin .$cu"; exit 0; }
    mv -v "$f" "${f%.$cu}.$moi"
done

chmod +x doiduoi.sh
./doiduoi.sh data txt bak
ls data
```

Ghi chú: `${f%.$cu}` cắt phần đuôi `.txt` khỏi cuối tên tập tin.

Ví dụ 5.3: Sao lưu thư mục có gắn ngày giờ

```
#!/bin/bash
# sao_luu.sh
nguon="$1"
dich="${2:-$HOME/backup}"

if [ ! -d "$nguon" ]; then
    echo "Thu muc nguon khong ton tai: $nguon"
    exit 1
fi

[ -d "$dich" ] || mkdir -p "$dich"

tenfile="$dich/$(basename "$nguon")_$(date +%Y%m%d_%H%M%S).tar.gz"

if tar -czf "$tenfile" -C "$(dirname "$nguon")" "$(basename "$nguon")"; then
    echo "Sao luu thanh cong: $tenfile"
    ls -lh "$tenfile"
else
    echo "Sao luu that bai"
    exit 2
fi

chmod +x sao_luu.sh
./sao_luu.sh ~/lab_shell/data
ls ~/backup
```

Ví dụ 5.4: Tìm và xóa tập tin cũ (có xác nhận)

```
#!/bin/bash
# xoa_cu.sh: liet ke va xoa file .log cu hon N ngay
thumuc="$1"
ngay="${2:-7}"

[ -d "$thumuc" ] || { echo "Thu muc khong ton tai"; exit 1; }

echo "Cac tap tin .log cu hon $ngay ngay trong $thumuc:"
find "$thumuc" -type f -name "*.log" -mtime +"$ngay"

read -p "Ban co muon xoa khong? (y/n): " tl
if [ "$tl" = "y" ]; then
    find "$thumuc" -type f -name "*.log" -mtime +"$ngay" -delete
    echo "Da xoa"
else
    echo "Huy bo"
fi
```

Để thử, tạo tập tin giả cũ 10 ngày: `touch -d "10 days ago" data/cu.log`.

> **Cảnh báo:** Luôn liệt kê trước khi xóa. Không chạy các script xóa với quyền `root` trên đường dẫn chưa kiểm tra.

Ví dụ 5.5: Phân loại tập tin theo phần mở rộng

```
#!/bin/bash
# phanloai.sh: chuyen file vao thu muc theo duoi
thumuc="${1:-.}"

for f in "$thumuc"/*.*; do
    [ -f "$f" ] || continue
    duoi="${f##*.}"
    mkdir -p "$thumuc/$duoi"
    mv -v "$f" "$thumuc/$duoi/"
done
```

Ví dụ 5.6: Kiểm tra máy trong mạng còn hoạt động (ứng dụng quản trị mạng)

Tạo tập tin `danhsach.txt` mỗi dòng một địa chỉ hoặc tên máy:

```
printf "127.0.0.1\n8.8.8.8\ndebian.org\n192.168.99.99\n" > danhsach.txt

#!/bin/bash
# ping_list.sh
ds="${1:-danhsach.txt}"
[ -f "$ds" ] || { echo "Khong tim thay $ds"; exit 1; }

while read -r host; do
    [ -z "$host" ] && continue
    if ping -c 1 -W 2 "$host" > /dev/null 2>&1; then
        echo "[ UP   ] $host"
    else
        echo "[ DOWN ] $host"
    fi
done < "$ds"
```

Bài tập tự làm

1. Viết `chanle.sh` nhận một số nguyên từ tham số, in ra số đó là chẵn hay lẻ; báo lỗi nếu không truyền tham số.
2. Viết `giaithua.sh` tính n! (n nhập từ bàn phím) bằng vòng lặp `for`.
3. Viết `songuyento.sh` kiểm tra một số có phải số nguyên tố hay không.
4. Viết `thongtin_file.sh` nhận nhiều đường dẫn làm tham số (`$@`), với mỗi đường dẫn in ra: tồn tại hay không, là tập tin hay thư mục, và kích thước nếu là tập tin (gợi ý: `stat -c %s` hoặc `wc -c`).
5. Viết `dem_dai.sh` đếm số tập tin theo từng phần mở rộng trong một thư mục (ví dụ: `txt: 5`, `jpg: 3`).
6. Viết `ktra_dichvu.sh` nhận tên dịch vụ (ví dụ `ssh`, `cron`) và dùng `systemctl is-active` để thông báo dịch vụ đang chạy hay đã dừng.

7. **Nâng cao:** Viết script tạo hàng loạt thư mục `lop01` … `lop10`, trong mỗi thư mục tạo 5 tập tin `sv01.txt` … `sv05.txt`; sau đó viết script thứ hai xóa toàn bộ cấu trúc này (có xác nhận).

Gỡ lỗi script

```
bash -n ten_script.sh     # kiem tra loi cu phap, khong chay
bash -x ten_script.sh     # chay va in tung lenh khi thuc thi
```

Các lỗi thường gặp:

Lỗi	Nguyên nhân thường gặp
`Permission denied`	Chưa `chmod +x`
`command not found`	Sai tên lệnh, hoặc gọi script mà thiếu `./`
`bad interpreter` / `^M`	Tập tin có ký tự xuống dòng kiểu Windows; sửa bằng `sed -i 's/\r$//' file.sh`
`[: missing ']'`	Thiếu khoảng trắng quanh `[` và `]`
`unary operator expected`	Biến rỗng không đặt trong dấu nháy kép
`command not found` khi gán biến	Có khoảng trắng quanh dấu `=` (viết `a=5`, không viết `a = 5`)

Tổng kết

- Script bắt đầu bằng `#!/bin/bash`, cần quyền `x` để chạy trực tiếp.
- Biến gán không có khoảng trắng quanh `=`, dùng `$((...))` cho số học số nguyên.
- Điều kiện dùng `[ ]` hoặc `[[ ]]`; kết hợp bằng `&&`, `||`, `!`.
- Kiểm tra tập tin: `-e`, `-f`, `-d`, `-r`, `-w`, `-x`, `-s`.
- Vòng lặp: `for`, `while`, `until`, cùng `break` và `continue`.
- Luôn đặt biến trong dấu nháy kép khi dùng với tên tập tin để tránh lỗi với tên có khoảng trắng.